TURNI · DOCUMENTAZIONE TECNICA

Il motore di calcolo

Paga lorda e netta secondo la busta paga italiana: architettura, guida, riferimento, impostazioni, regole e riscontri

VERSIONE DEL MOTORE	1.0.0
DATA	12 ottobre 2026
CARTELLA	`src/motore/`
RISCONTRI	19 del motore · 47 di Turni
STATO	BETA: stime da far controllare a un professionista

Indice

1. Panoramica

Che cos'è

Tre regole che il motore rispetta

La struttura

Il flusso di un mese

2. Guida

Come si usa

Cosa sta dove

Aggiungere o togliere, a mano

Tre regole che il motore rispetta

I riscontri

Più a fondo

Una cosa che resta di Turni

3. Riferimento delle funzioni

4. Le impostazioni

5. La macchina

0. In una frase

1. Mappa dei file

2. Modello dati

3. La catena, dall'alto

4. Fase 1 — Durata di un turno

5. Fase 2 — Raggruppamento: dove si cumulano le ore

6. Fase 3 — Supplementare e straordinario

7. Fase 4 — Le maggiorazioni

8. Fase 5 — Assenze

9. Fase 6 — L'output di computePayByShift

10. Fase 7 — Dal lordo del mese al reddito annuo di riferimento

11. Fase 8 — calcNetAnnual(grossAnnual, settings)

12. Fase 9 — calcNetMonthly(...) — la busta del mese

13. Cosa è verificato e cosa no

14. Come si verifica una modifica

15. Trappole note

16. Due grandezze che possono legittimamente non coincidere

17. Il gate beta

18. Se devi estendere il motore

6. I riscontri

Del motore — src/motore/riscontri

Di Turni — scripts/

PARTE 1

Panoramica

Che cos'è il motore, come è fatto, le regole che non si rompono

Che cos'è

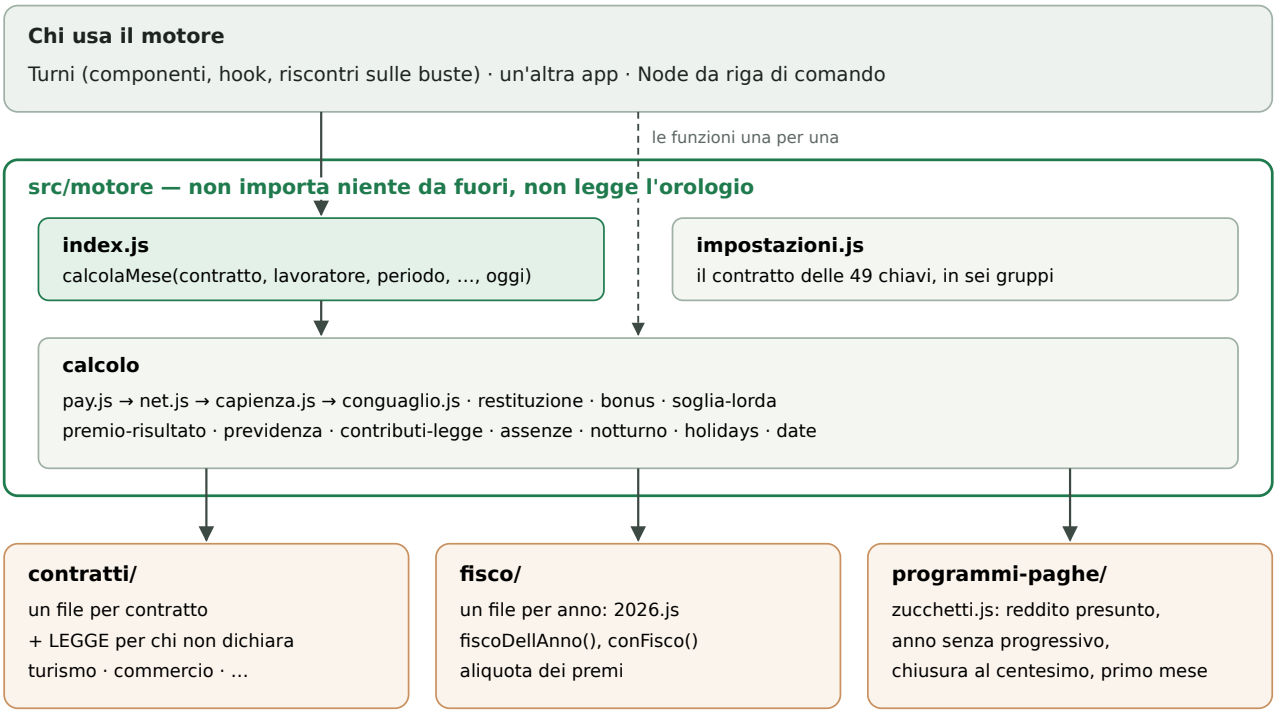
Il motore calcola paga lorda e netta secondo la busta paga italiana: dai turni timbrati alle ore ordinarie, supplementari, straordinarie e maggiorate; dal lordo ai contributi, all'IRPEF, alle detrazioni, al trattamento integrativo e all'indennità L. 207/24; dal mese all'anno, fino al conguaglio di dicembre. Vive in `src/motore/` e non importa niente dal resto di Turni: si copia in un'altra applicazione e si verifica da solo con `npm test`.

Ogni regola che il motore dà per buona ha il suo riscontro: o una busta paga reale da cui è stata ricavata, o la norma che la fissa. Il motore è marcato BETA: le sue stime vanno fatte controllare da un professionista.

Tre regole che il motore rispetta

1. **Non importa niente da fuori dalla sua cartella.** Niente pacchetti npm, niente `window`, `localStorage` o `fetch`. Lo controlla `check-confine-motore.mjs`.
2. **Non legge l'orologio.** La data di oggi la passa chi chiama; senza, le funzioni che ne hanno bisogno si fermano con un errore che lo dice (`richiediOggi`).
3. **Nessun numero cambia senza un riscontro.** `check-numeri-fermi.mjs` blocca al centesimo l'uscita su cinque profili di contratto per dodici mesi: uno spostamento di codice non deve muoverla.

La struttura



SRC/MOTORE/	RIGHE	COSA CONTIENE
`index.js`	101	la porta: calcolaMese(), impostazioniDa(), i pezzi

SRC/MOTORE/	RIGHE	COSA CONTIENE
impostazioni.js	81	le 49 chiavi che il motore legge, in sei gruppi
pay.js	548	turni → lordo: monte ore, supplementare, straordinario, maggiorazioni
net.js	1021	lordo → netto: contributi, IRPEF, detrazioni, TI, cuneo, proiezioni
capienza.js	79	capienza del TI sul progressivo, reddito presunto
conguaglio.js	356	i dodici mesi, il conguaglio di dicembre
restituzione.js	391	rischio di restituzione del TI
bonus.js	118	margini rispetto alle soglie
soglia-lorda.js	46	soglie di reddito tradotte in lordo
premio-risultato.js	60	premio di risultato a imposta sostitutiva
previdenza.js	58	fondo pensione e cassa sanitaria
contributi-legge.js	76	FIS e CIGS per dimensione d'azienda
assenze.js	163	ferie, permessi, malattia, festività
periodo-assenza.js	138	ferie a periodo, riposi e festività esclusi
notturno.js	272	fascia notturna, straordinario notturno
holidays.js	72	festività nazionali, Pasqua
date.js	109	aritmetica di calendario, richiediOggi()
mensilita.js	5	i mesi di 13ª e 14ª
ccnl.js	66	lettura dei contratti
contratti/		index.js + turismo, commercio, commercio-confesercenti, vigilanza, domestico
fisco/		index.js + 2026.js
programmi-paghe/		index.js + zucchetti.js
riscontri/		19 riscontri delle regole, esegui.mjs, numeri-fermi.json
LEGGIMI.md · MANUALE.md · package.json		guida, manuale, pacchetto 1.0.0

Il flusso di un mese

```
turni dell'anno + impostazioni + oggi
   │
   ├─ computePayByShift      pay.js       paga di ogni turno (venti campi)
   ├─ calcTotalPay           pay.js       lordo dei turni del mese, voce per voce
   ├─ lordoDelMese           net.js       + voci fisse, premio, 13ª/14ª del mese
   ├─ progressiviDellAnno    conguaglio   lordo e mesi dell'anno prima di questo
   ├─ capienzaProgressiva    capienza     reddito presunto (programma paghe) e capienza del TI
   └─ nettoDelMese           net.js       contributi, IRPEF, detrazioni, TI, indennità, netto
                                          col fisco dell'anno (conFisco)
```

`calcolaMese()` in `index.js` fa questa catena per un mese, ricevendo gli ingressi divisi per natura: contratto, lavoratore, periodo, programma paghe, data di oggi.

PARTE 2

Guida

Come si usa e come si modifica a mano

Calcola paga lorda e netta secondo la busta paga italiana: turni, maggiorazioni, contributi, IRPEF, detrazioni, trattamento integrativo, conguaglio di dicembre. Questa cartella non importa niente da fuori (né React, né il browser, né il resto dell'app) e si può copiare così com'è in un altro progetto.

Come si usa

```
import { calcolaMese } from './motore/index.js';

const { lordo, netto } = calcolaMese({
  contratto: 'turismo',
  lavoratore: {
    pagaOraria: 9.21802, oreSettimanali: 24, assunzione: '2025-12-29',
    tredicesima: true, quattordicesima: true,
    maggiorazioni: { supplementare: 30, domenicale: 10, festivo: 20 },
  },
  periodo: { anno: 2026, mese: 8, turni },   // mese da 0 a 11
  oggi: new Date(),
});
```

`turni` è un elenco di `{ id, date: 'YYYY-MM-DD', startTime: 'HH:MM', endTime: 'HH:MM', breakMinutes }`; ferie, permessi, malattia e festività hanno `type` e `durationMinutes` al posto degli orari. Servono i turni di tutto l'anno: le settimane a cavallo dei mesi e il progressivo dell'anno li usano.

La tabella che traduce questi ingressi nei campi interni è `impostazioniDa()` in `index.js`. Le funzioni una per una (paga per turno, proiezione dell'anno, conguaglio) sono esportate dallo stesso file.

Cosa sta dove

CARTELLA O FILE	COSA CONTIENE	CHI LA CAMBIA
`contratti/`	un file per contratto: divisore orario, mensilizzazione, Ente Bilaterale, fascia notturna, straordinario notturno	chi aggiunge o corregge un CCNL
`fisco/`	un file per anno: scaglioni IRPEF, detrazioni, trattamento integrativo, indennità L. 207/24; l'aliquota dei premi di risultato	a ogni legge di bilancio
`programmi-paghe/`	come il programma paghe del datore anticipa l'anno mese per mese (reddito presunto)	chi trova un datore che fa diversamente
`holidays.js`	le festività nazionali, una riga ciascuna	quando una legge ne aggiunge una
`pay.js`	dalla timbratura agli euro lordi: monte ore, supplementare, straordinario, maggiorazioni	
`net.js`	dal lordo al netto: contributi, IRPEF, detrazioni, trattamento integrativo	

CARTELLA O FILE	COSA CONTIENE	CHI LA CAMBIA
`capienza.js`, `conguaglio.js`, `restituzione.js`	il trattamento integrativo mese per mese e a dicembre	
`impostazioni.js`	ogni chiave delle impostazioni, col suo gruppo	chi aggiunge un'impostazione
`riscontri/`	i riscontri delle regole, con dati sintetici	

Aggiungere o togliere, a mano

Un contratto. Copia `contratti/turismo.js` in `contratti/<codice>.js`, cambia `codice`, `nome` e le regole, e aggiungi una riga in `contratti/index.js` (l'`import` e il nome nell'elenco). Un campo che non scrivi vale la regola di legge (`LEGGE` nello stesso file). Per toglierlo: cancella la riga e il file.

Un anno fiscale. Copia `fisco/2026.js` in un file nuovo (2027.js), cambia `anno` e i valori che la legge di bilancio cambia, aggiungi la riga in `fisco/index.js`. I calcoli che conoscono l'anno lo useranno da soli.

Un programma paghe. Copia `programmi-paghe/zucchetti.js`, cambia le regole tenendo le stesse quattro voci (`lordoPresuntoAnno`, `annoSenzaProgressivo`, `quota`, `primoMeseConcesso`), aggiungi la riga in `programmi-paghe/index.js`. Si sceglie con `programmaPaghe`; senza, vale Zucchetti, l'unico verificato sulle buste.

Un'impostazione nuova. Una riga in `impostazioni.js`, col suo gruppo e cosa vuol dire. Il riscontro `check-impostazioni-motore.mjs` diventa rosso se il motore legge una chiave che lì non c'è.

Una festività. Una riga in `holidays.js`: in `FIXED` se vale da sempre, in `DAL_ANNO` col primo anno se è nuova.

Tre regole che il motore rispetta

1. **Non importa niente da fuori da questa cartella.** Niente pacchetti npm, niente `window`, `localStorage` o `fetch`.
2. **Non legge l'orologio.** La data di oggi la passa chi chiama; senza, le funzioni che ne hanno bisogno si fermano con un errore che lo dice.
3. **Nessun numero cambia senza un riscontro.** Ogni regola ha il suo `riscontri/check-*.mjs`; `check-numeri-fermi.mjs` blocca al centesimo l'uscita su cinque profili, e uno spostamento non deve muoverla.

Le prime due le controlla `riscontri/check-confine-motore.mjs`.

I riscontri

```
npm test                                  # dalla cartella del motore: tutti i suoi riscontri
node riscontri/check-numeri-fermi.mjs     # i numeri non si sono mossi
node riscontri/check-confine-motore.mjs   # il motore non esce dalla cartella
```

Quelli sulle buste paga di una persona non stanno qui: restano all'app che li ha (in Turni, `scripts/check-busta-*.mjs` e simili), perché contengono le sue cifre.

Più a fondo

`MANUALE.md` descrive la macchina: la catena dai turni al netto, cosa è verificato su una busta e cosa no, le trappole note.

Una cosa che resta di Turni

I nomi delle impostazioni (`hourlyRate`, `expectedWeeklyHours`…) sono quelli che Turni salva nel telefono di chi lo usa, e non si cambiano: rinominarli voleva dire migrare i dati di tutti, per un motore che non diventa più portabile per questo. Sono tutti in `impostazioni.js`, col loro significato; `calcolaMese` li nasconde dietro nomi italiani.

PARTE 3

Riferimento delle funzioni

Le funzioni esportate, modulo per modulo, lette dal codice

Ogni riga riporta la firma e la prima frase del commento che la precede nel sorgente. Per il comportamento completo vale il codice, con i suoi riscontri.

`index.js` — La porta d'ingresso

`impostazioniDa({ contratto = '', lavoratore = {}, programmaPaghe, altro = {} })`	Dagli ingressi del motore alle impostazioni che le sue funzioni leggono (tutte descritte in impostazioni.js).
`calcolaMese({ periodo, oggi, ...ingressi })`	Quanto vale un mese:

`pay.js` — Dalla timbratura al lordo

`calcShiftMinutes(shift)`	Ferie, permessi e malattia non hanno orari:
`parseNum(v)`	Parsing robusto di numeri all'italiana:
`getRateForDate(dateStr, settings)`	Paga oraria valida per una certa data.
`hasAnyRate(settings)`	Esiste almeno una paga oraria configurata (attuale o precedente)?
`isSunday(dateStr)`	
`getShiftSurchargeParts(shift, settings)`	Maggiorazioni di un turno, tenute DISTINTE per tipo (percentuali):
`getShiftSurchargePct(shift, settings)`	Percentuale di maggiorazione totale per un turno:
`computePayByShift(allShifts, settings)`	Serve l'insieme completo dei turni per raggruppare correttamente.
`lordoTurno(voce)`	Totale paga con dettaglio maggiorazioni.
`calcTotalPay(shifts, settings, allShifts = shifts, byShift = null)`	
`formatCurrency(amount)`	`useGrouping:

`net.js` — Dal lordo al netto

`irpefLorda(imponibile, T = FISCO_ULTIMO)`	
`aliquotaMarginale(imponibile, T = FISCO_ULTIMO)`	Aliquota dello scaglione in cui cade il reddito:
`calcContributi(gross, settings = {}, ebBase = 0, mesi = 1)`	Contributi a carico del dipendente, voce per voce come in busta paga.
`detrazioneLavoro(reddito, T = FISCO_ULTIMO)`	
`cuneoPercent(redditoComplessivo, redditoLavoro = redditoComplessivo, T = FISCO_ULTIMO)`	Percentuale della somma "cuneo" (L.
`trattamentoIntegrativo(reddito, irpef, detLavoro, T = FISCO_ULTIMO)`	Trattamento integrativo, calcolato sull'imponibile e sulle detrazioni.
`deductibleContribRate(settings = {})`	Aliquota contributiva DEDUCIBILE totale (IVS + contributi minori del CCNL).

`redditoComplessivo(gross, settings = {}, contributi = null)`	Reddito complessivo ai fini IRPEF — la grandezza su cui la legge misura le soglie del trattamento integrativo (15.000 / 28.000).
`tiSpettaQuestoMese(lordoMese, settings = {}, lordoPresunto = null)`	Il trattamento integrativo spetta in QUESTO mese? La legge guarda l'anno, la busta il mese:
`modoTrattamentoIntegrativo(settings = {})`	'auto' (come il datore) · 'sempre' · 'mai'.
`tiSospeso(settings = {})`	Il bonus è stato fatto sospendere dal datore (a mano, in una delle due schermate).
`patchTiSospeso(sospeso)`	La modifica da scrivere per sospendere o riattivare il bonus:
`riferimentoAnnuoDelMese(lordoMese, settings = {}, lordoPresunto = null)`	Il reddito annuo di riferimento che il software paghe usa PER QUESTO MESE:
`lordoDelMese(pagaTurni, anno, mese, settings = {}, { enableNetCalc = true } = {})`	Il lordo di UN mese, come lo mostra l'app:
`nettoDelMese(lordoMese, settings, giorniMese, extraMese = 0, capienza = null)`	Il netto di UN mese, come lo mostra l'app:
`taxableToGross(taxable, settings = {})`	
`monthlyBaseGross(settings = {})`	Retribuzione mensile "base" da contratto:
`extraMonthAccrual(kind, year, settings = {})`	Frazione di mensilità aggiuntiva effettivamente maturata (0..1).
`receivedExtraMonthsCount(settings, monthIndex, year)`	Mensilità aggiuntive GIÀ arrivate entro il mese indicato (indice 0-11), espresse in mensilità equivalenti:
`extraMonthsAccrued(settings, year)`	Mensilità aggiuntive previste nell'INTERO anno, in mensilità equivalenti.
`computeAnnualGrossFromShifts(year, allShifts, settings = {}, payByShift = null, oggi)`	Reddito annuo lordo MATURATO nell'anno indicato:
`projectAnnualIncome(annualGross, annualExtras, settings = {}, year, { enableNetCalc = true, oggi } = {},)`	
`calcNetAnnual(grossAnnual, settings = {}, { giorni = 365 } = {})`	Calcola la stima del netto annuo.
`tiDecision(annualGrossRef, settings = {})`	Decisione automatica sul trattamento integrativo (come un software paghe), data la proiezione di reddito annuo `annualGrossRef`.
`calcNetMonthly(monthGross, annualGrossRef, settings = {}, monthDays = 365 / 12, extraMonthGross = 0, capienza = null)`	Stima del netto del MESE, partendo dal lordo mensile — come una busta paga.

`capienza.js` — Capienza e reddito presunto

`capienzaProgressiva(lordoMese, settings = {}, progressivo = null)`	

`lordoPresuntoAnno(lordoMese, settings = {}, progressivo = null)`	Il LORDO PRESUNTO dell'anno, su cui il datore decide ogni mese trattamento integrativo e detrazioni.
`nettoSimulandoTi(lordoMese, settings, giorniMese, extraMese, capienza, presente)`	Il netto del mese col tratt.
`simulaTi(vero, giorniMese, presente, impostazioni = {})`	Lo stesso, partendo da un netto già calcolato (anche col premio di risultato).

`conguaglio.js` — L'anno e il conguaglio

`saldoConguaglio(mesi, impostazioni = {}, { tiMontanteNoto = null, irpefMontanteNota = null, anno } = {})`	Il saldo del conguaglio su dodici mesi dati.
`mesiDellAnno({ anno, allShifts = [], settings = {}, payMap, oggi, scenario = 'contratto', montanteAlterno = false, totaleAnno = null, })`	I dodici mesi dell'anno come li vede l'app, in uno dei due scenari per i mesi che restano.
`progressiviDellAnno(args)`	Il progressivo dell'anno PRIMA di ogni mese — lordo e mesi con un lordo — dagli stessi mesi del conguaglio (montante diviso, turni, poi il contratto).
`stimaConguaglio({ anno, allShifts, settings, payMap, oggi, proiezioneAnnua = null })`	La stima da mostrare:

`restituzione.js` — Il rischio di restituzione

`giorniTrascorsi(data)`	Giorni dell'anno già trascorsi alla data indicata, estremi compresi.
`dataDiRiferimento(anno, oggi)`	Il giorno a cui misurare l'anno GUARDATO, che non è sempre quello di oggi:
`quotaPotenziale(oggi)`	Quanto avresti già preso SE il datore ti stesse accreditando la quota piena fin qui — l'ipotesi che serve all'anteprima «sei vicino alla soglia, se la superi restituisci circa X».
`rischioRestituzione({ settings: impostazioni = {}, proiezioneAnnua = 0, oggi, erogatoStimato = null } = {})`	Il rischio di restituzione, in euro.
`costoSoglia(settings = {})`	La forma della buca attorno ai 15.000 €, calcolata sul motore.
`posizioneRispettoSoglia(proiezioneAnnua, settings = {}, costo = null)`	Dove si trova il reddito previsto rispetto alla buca.
`mancaAlPareggio(proiezioneAnnua, settings = {}, costo = null)`	Quanto manca, in euro di lordo, per uscire dalla buca dal basso.
`confrontoConSoglia(lordo, settings = {}, costo = null)`	Cosa succede AL TUO LORDO, non nel punto peggiore.

`bonus.js` — Margini rispetto alle soglie

`margineInOre(margineLordo, settings = {})`	Calcola i margini rispetto alle soglie del bonus.
`calcBonusMargin(annualIncome, settings = {})`	

`soglia-lorda.js` — Soglie in lordo

`lordoPerReddito(reddito, settings = {})`	Il lordo annuo più alto che resta entro un REDDITO dato:
`lordoMeseMinimoTi(settings = {})`	Il lordo del MESE da cui il trattamento integrativo arriva in busta.

`premio-risultato.js` — Premio di risultato

`nettoDelMeseConPremio(mese, impostazioni, giorniMese, anno, capienza = null)`	Il netto del mese col premio di risultato tassato a parte.

`previdenza.js` — Fondo pensione e cassa sanitaria

`trattenuteFuoriReddito(gross, settings = {}, mesi = 1)`	Le righe di trattenuta del periodo, nello stesso formato di `calcContributi`.

`contributi-legge.js` — FIS e CIGS

`contributiDiLegge(settings = {}, schema = null)`	Contributi di legge a carico del LAVORATORE, in percentuale sull'imponibile previdenziale.

`assenze.js` — Assenze

`tipoTurno(shift)`	I turni salvati prima di questa funzione non hanno `type`:
`isAssenza(shift)`	
`minutiGiornoAssenza(settings = {})`	Ore che vale un giorno intero di assenza, in minuti.
`giorniEventoMalattia(allShifts)`	Posizione (a partire da 1) di ogni giorno di malattia dentro il PROPRIO evento.
`percentualeAssenza(shift, settings = {}, giornoEvento = 0)`	Percentuale della retribuzione oraria che spetta per un'assenza.

`periodo-assenza.js` — Ferie a periodo

`giorniDiRiposo(turni = [], settings = {})`	I giorni della settimana (0 = domenica) di riposo:
`giorniPeriodo(dal, al)`	Le date ISO comprese fra due estremi, estremi inclusi.
`proponiPeriodo({ dal, al, turni = [], settings = {} } = {})`	Le giornate proposte per un periodo di assenza, da mostrare e correggere prima di salvare.
`totalePeriodo(righe)`	Quante giornate e quanti minuti valgono le righe spuntate.

`notturno.js` — Lavoro notturno

`fasciaNotturnaRisolta(settings = {})`	Fascia che vale per queste impostazioni, come testo:
`fasciaNotturnaPossibile(settings = {})`	
`toccaFasciaNotturnaPossibile(shift, settings = {})`	Questo turno tocca la fascia in cui la notte POTREBBE cadere.
`fasciaNotturna(settings = {})`	Estremi della fascia in minuti dalla mezzanotte, più la sua durata.
`minutiInFasciaNotturna(startTime, endTime, settings = {})`	Minuti del turno che cadono nella fascia notturna.
`minutiNotturni(shift, settings = {})`	Come sopra, ma a partire da un turno.
`minutiNotturniPagati(shift, settings = {}, minutiPagati = 0)`	Minuti notturni che si possono davvero PAGARE:
`toccaFasciaNotturna(shift, settings = {})`	Il turno tocca la fascia? Serve alla UI per l'etichetta «notturno», e deriva dal conteggio invece di duplicarne la regola:
`pctNotturno(settings = {})`	Percentuale di maggiorazione notturna impostata dall'utente (0 = spenta).
`modoCumuloNotturno(settings = {})`	'max' (la maggiore assorbe la minore) oppure 'somma'.
`pctNotturnoAggiuntiva(settings = {}, pctGiorno = 0)`	Percentuale AGGIUNTIVA che spetta ai minuti notturni, oltre a quella che il turno prende già per domenica/festivo/manuale.
`straordinarioNotturno(shift, minutiStraordinari, settings = {})`	Lo STRAORDINARIO NOTTURNO del contratto:

`holidays.js` — Festività

`easterSunday(year)`	Domenica di Pasqua (computus, algoritmo di Meeus/Gauss).
`isHoliday(dateStr, settings = {})`	True se la data ISO 'YYYY-MM-DD' è una festività (nazionale, Pasqua/Pasquetta, o santo patrono locale se impostato in settings.patronSaintDate come 'MM-DD').

`date.js` — Date e orologio

`getWeekStart(date)`	Le date del motore:
`formatDate(date)`	
`parseDate(str)`	
`dayNumber(iso)`	L'app usa ovunque la stringa ISO 'YYYY-MM-DD' come chiave e come criterio di ordinamento (slice per anno/mese, confronti lessicografici).
`isIsoDate(str)`	

`parseTime(value)`	'HH:MM' → minuti dalla mezzanotte, oppure null se non è un orario valido.
`minutesDiff(startTime, endTime)`	
`payrollMonthKey(dateStr)`	Mese di paga di una data, come 'YYYY-MM'.
`payrollMonthRange(year, month)`	Estremi del mese di paga:
`getDaysInMonth(year, month)`	
`richiediOggi(oggi, chi)`	IL MOTORE NON LEGGE L'OROLOGIO:

`ccnl.js` — Lettura dei contratti

`getCcnl(codice)`	Le regole del contratto `codice`; un codice sconosciuto vale la legge.
`monthlyHoursFactor(settings = {})`	Fattore ore mensili del contratto scelto (settimane retribuite al mese).
`isMensilizzato(settings = {})`	Contratto MENSILIZZATO:
`monthlyContractHours(settings = {})`	Ore contrattuali di un mese:
`monthlyFullTimeHours(settings = {})`	Ore FULL-TIME di un mese, stesso fattore di conversione delle ore contrattuali:

`fisco/index.js` — Fisco per anno

`fiscoDellAnno(anno)`	La tabella dell'anno, o quella dell'ultimo anno noto prima di lui.
`fiscoDi(impostazioni)`	La tabella delle impostazioni del calcolo, o l'ultima nota.
`conFisco(impostazioni, anno)`	Le impostazioni del calcolo con la tabella dell'anno `anno`.
`aliquotaPremio(anno)`	Aliquota dell'imposta sostitutiva sui premi di risultato nell'anno.

`programmi-paghe/index.js` — Programmi paghe

`programmaPaghe(settings = {})`	Il programma paghe delle impostazioni, o quello verificato sulle buste.

PARTE 4

Le impostazioni

Le 49 chiavi che il motore legge, da impostazioni.js

Le funzioni ricevono un oggetto piatto. I nomi sono quelli che Turni salva nel telefono di chi lo usa, e per questo non si cambiano; `calcolaMese()` li nasconde dietro nomi italiani. `check-impostazioni-motore.mjs` confronta questa tabella col codice in tutte e due le direzioni. Un campo vuoto (`''`) significa «calcolalo tu», non zero.

Il contratto 2

`ccnl`	codice di un file in contratti/; vuoto = la regola di legge
`periodoConteggio`	'calendario' (verificato) o 'paga' (settimane intere): il mese su cui si conta il monte ore

Il lavoratore 34

`hourlyRate`	€ lordi all'ora, la paga di oggi
`previousRates`	paghe precedenti a un aumento: [{ until: 'YYYY-MM-DD', rate }]
`expectedWeeklyHours`	ore settimanali del contratto
`fullTimeWeeklyHours`	ore del tempo pieno: oltre, straordinario
`workingDaysPerWeek`	giorni lavorativi della settimana: quanto vale una giornata di assenza
`onCall`	lavoro a chiamata: nessun monte ore, soglia giornaliera
`dailyOvertimeThreshold`	a chiamata: ore al giorno oltre cui c'è la maggiorazione
`hireDate`	data di assunzione, 'YYYY-MM-DD': ratei di 13ª/14ª, giorni di detrazione
`hasTredicesima`	la 13ª spetta
`hasQuattordicesima`	la 14ª spetta
`overtimeSurchargePct`	% in più sul supplementare
`straordinarioSurchargePct`	% in più sullo straordinario; vuoto = come il supplementare
`sundaySurchargePct`	% in più la domenica
`holidaySurchargePct`	% in più nei festivi
`holidaySundayMode`	festivo di domenica: 'max' (la maggiore assorbe), 'sum', 'holiday'
`nightSurchargePct`	% in più sulle ore notturne
`nightStart`	inizio della fascia notturna 'HH:MM'; vuoto = quella del contratto
`nightEnd`	fine della fascia notturna 'HH:MM'; vuoto = quella del contratto
`nightCumuloMode`	notturno con altre maggiorazioni: 'max' (assorbe) o 'somma'
`patronSaintDate`	santo patrono del comune, 'MM-DD': festivo come gli altri
`absenceDailyHours`	ore di una giornata di ferie o permesso; vuoto = ore settimanali ÷ giorni
`malattiaCarenzaGiorni`	giorni di carenza della malattia, per evento (non verificati)
`malattiaCarenzaPct`	% pagata nei giorni di carenza (non verificata)

`malattiaPct`	% pagata dopo la carenza (non verificata)
`aziendaDipendenti`	dimensione dell'azienda: decide le aliquote del FIS
`ebtBase`	base dell'Ente Bilaterale letta in busta, se diversa da paga × monte ore
`addRegionalePct`	% dell'addizionale regionale
`addComunalePct`	% dell'addizionale comunale
`addizionaliAltrove`	le addizionali le trattiene un altro datore
`noAddizionali`	primo anno di lavoro: nessuna addizionale da trattenere
`fondoPensionePct`	% della quota a carico del lavoratore per il fondo pensione
`cassaSanitariaEuro`	€ al mese per la cassa sanitaria trattenuti in busta
`tfrInBusta`	il TFR è pagato ogni mese in busta
`tfrTaxRate`	% di tassazione del TFR in busta; vuoto = il predefinito

Il periodo 8

`priorTaxableIncome`	montante: lordo dell'anno già pagato fino a priorIncomeDate
`priorIncomeDate`	fino a quando arriva il montante, 'YYYY-MM-DD'
`monthlyBonusAmount`	€ del premio mensile, quando c'è
`monthlyBonus`	i mesi col premio: { 'YYYY-MM': true }
`bonusPremioRisultato`	il premio è di risultato: imposta sostitutiva invece dell'IRPEF
`fixedMonthlyItems`	voci fisse in busta ogni mese: [{ amount }]
`fixedMonthlyDeductions`	trattenute fisse ogni mese, fuori dal fisco: [{ amount }]
`annualGrossManual`	lordo dell'anno scritto a mano, per la proiezione

Il programma paghe 1

`programmaPaghe`	codice di un file in programmi-paghe/; vuoto = Zucchetti, il verificato

Il fisco 1

`fisco`	la tabella dell'anno (fisco/), messa da chi conosce l'anno con conFisco()

Le scelte di calcolo 3

`tiModo`	trattamento integrativo: 'auto' (come il datore), 'sempre', 'mai'
`noTrattamentoIntegrativo`	storico: tiModo 'mai' scritto alla vecchia maniera
`tiProjectionMode`	proiezione dell'anno: 'stimato' o 'ytd' (solo il maturato)

PARTE 5

La macchina

Il manuale: come i fatti diventano un calcolo, in che ordine, cosa si rompe

Documento di riferimento per chi (persona o modello) deve **modificare, estendere o riscrivere** il calcolo di ore, lordo e netto. Non è una guida all'uso dell'app: è la descrizione della macchina che sta sotto.

Aggiornato al 12 ottobre 2026. La guida rapida — come si usa, come si aggiunge un contratto, un anno o un programma paghe — è `LEGGIMI.md`; questo è il manuale della macchina.

> **Rapporto con `CLAUDE.md` di Turni.** Quello è l'orientamento rapido: i fatti verificati sulle buste, in forma di elenco, con il riscontro accanto a ciascuno. Questo descrive la **macchina**: come i fatti diventano un calcolo, in che ordine, e cosa si rompe toccandoli. Dove i due si sovrappongono, **`CLAUDE.md` è la fonte**: è il file che si aggiorna scoprendo qualcosa di nuovo su una busta. Se divergono, si allinea questo.

---

0. In una frase

Il motore prende un insieme di **turni** (`{date, startTime, endTime, ...}`) e un oggetto **impostazioni** (contratto, paga oraria, percentuali: tutte in `impostazioni.js`) e produce, attraverso una catena di funzioni pure, tre grandezze: **ore**, **lordo** e **netto stimato del mese** — quest'ultimo ricalcando la struttura di una busta paga italiana 2026.

Tutto è **puro e sincrono**. Nessuna funzione del motore legge da `localStorage`, dalla rete, da React o dall'orologio: lo stato e la data di oggi entrano come argomenti. È la proprietà che permette di copiare la cartella in un'altra app, e di importare gli stessi moduli da Node puro per metterli a confronto con buste paga reali. La tiene `check-confine-motore.mjs`.

---

1. Mappa dei file

Il motore vive in `src/motore/` e non importa niente da fuori. Nessun modulo di `components/` contiene aritmetica: se un numero non torna, il bug è qui.

FILE	RESPONSABILITÀ
`index.js`	la porta: `calcolaMese()`, `impostazioniDa()`, i pezzi esportati
`impostazioni.js`	ogni chiave delle impostazioni che il motore legge, col suo gruppo
`pay.js`	**cuore**: durata turni, monte ore, supplementare e straordinario, maggiorazioni, lordo
`net.js`	**fisco**: contributi, IRPEF, detrazioni, TI, cuneo, TFR, proiezioni
`capienza.js`	capienza del TI sul progressivo, reddito presunto, simulazione del TI
`conguaglio.js`	i dodici mesi dell'anno, il conguaglio di dicembre
`restituzione.js`, `bonus.js`, `soglia-lorda.js`	rischio di restituzione, margini e soglie lorde del TI

FILE	RESPONSABILITÀ
`premio-risultato.js`, `previdenza.js`, `contributi-legge.js`	premio detassato, fondo pensione e cassa sanitaria, FIS/CIGS
`assenze.js`, `periodo-assenza.js`	ferie, permessi, malattia, festività; ferie a periodo
`notturno.js`, `holidays.js`	fascia notturna e straordinario notturno; festività italiane e Pasqua
`date.js`, `mensilita.js`, `ccnl.js`	aritmetica di calendario e `richiediOggi`; mesi di 13ª/14ª; lettura dei contratti
`contratti/`	un file per contratto, `LEGGE` per chi non dichiara
`fisco/`	un file per anno, `fiscoDellAnno`, `conFisco`, aliquota dei premi
`programmi-paghe/`	un file per programma paghe: reddito presunto, chiusura al centesimo
`riscontri/`	i riscontri delle regole, con dati sintetici, e i numeri fermi

Restano all'app di Turni, perché non calcolano: `utils/stats.js` (aggregazione annuale), `utils/maggiorazioni.js` (normalizza le % copiate dal cedolino), `utils/festivita-non-lavorate.js`, `utils/orari.js`, `utils/ccnl-elenco.js` (l'archivio CNEL dei nomi), `hooks/useMonthlyNet.js`.

Regola di dipendenza: `pay.js` importa `date`, `holidays`, `ccnl`, `assenze`, `notturno`. `net.js` importa `ccnl`, `contributi-legge`, `pay`, `date`, `fisco/`, `programmi-paghe/`. `notturno.js` **non** importa `pay` — la dipendenza sarebbe circolare, ed è il motivo per cui `minutiNotturniPagati()` riceve i minuti pagati come parametro invece di calcolarseli.

Tutti gli import interni portano l'estensione `.js` esplicita. Non è uno stile: senza, Node puro non importa i moduli e i riscontri smettono di partire.

---

2. Modello dati

2.1 Turno (`shift`)

Salvato in `localStorage` sotto `turni_shifts`, come **mappa** `{ id: shift }`.

```
{
  id: string,             // genId()
  date: 'YYYY-MM-DD',     // chiave e criterio di ordinamento in tutta l'app
  startTime: 'HH:MM',     // assente sulle assenze
  endTime: 'HH:MM',       // può essere < startTime: turno che valica la mezzanotte
  breakMinutes: number,   // pausa non retribuita
  durationMinutes: number,// ALTERNATIVA a start/end: usata dalle assenze
  type: 'lavoro' | 'ferie' | 'permesso' | 'malattia' | 'festivita',
  surchargePct: number,   // maggiorazione manuale su questo turno
  note: string,
}
```

Due invarianti da non rompere:

- **`type` assente significa `lavoro`.** I turni salvati prima che esistessero le assenze non hanno il campo: nessuna migrazione, il valore mancante ha già il significato giusto (`tipoTurno()`).

- **`durationMinutes` vince su `startTime`/`endTime`.** Ferie, permessi e malattia non hanno orari: portano una durata già in minuti, perché in busta valgono un numero fisso di ore e non un intervallo (`calcShiftMinutes()`).

2.2 Impostazioni (`settings`)

Un oggetto piatto. **Il suo contratto è `impostazioni.js`**: ogni chiave che il motore legge, una riga, col gruppo (contratto, lavoratore, periodo, programma paghe, fisco, scelte) e il significato. `check-impostazioni-motore.mjs` lo confronta col codice: una chiave letta e non scritta lì, o scritta e non più letta, fa diventare rosso il riscontro. Due chiavi non vengono dall'utente: `fisco` la mette chi conosce l'anno (`conFisco`), `programmaPaghe` sceglie un file di `programmi-paghe/`.

In Turni le impostazioni vivono in `localStorage` (`turni_settings`), fuse sui `DEFAULT_SETTINGS` di `src/App.jsx` a ogni avvio; i nomi sono i loro, e per questo non si cambiano.

Convenzione ricorrente: il campo **vuoto (`''`) significa «calcolalo tu»**, non zero. Vale per `straordinarioSurchargePct`, `tfrTaxRate`, `absenceDailyHours`, `nightStart`/`nightEnd`. Chi aggiunge un campo nuovo di questo tipo deve seguire lo stesso schema, altrimenti un utente che non tocca il campo cambia comportamento senza saperlo.

---

3. La catena, dall'alto

```
turni + settings
      │
      ├─► computePayByShift(allShifts, settings)      ← pay.js — O(N) su TUTTA la storia
      │      mappa { shiftId: { base, surcharge, ...20 campi } }
      │
      ├─► calcTotalPay(shiftsDelPeriodo, settings, allShifts, byShift)
      │      lordo del periodo, con le voci ancora separate
      │
      ├─► computeAnnualGrossFromShifts(year, allShifts, settings, byShift)
      │      { total, extras }  ← maturato nell'anno + montante + 13ª/14ª incassate
      │
      ├─► projectAnnualIncome(total, extras, settings, year, opts)
      │      { value, source, voci[] }  ← reddito annuo DI RIFERIMENTO
      │
      ├─► calcNetAnnual(valoreDiRiferimento, settings)
      │      IRPEF annua, detrazioni, TI, cuneo — la scala su cui si misura il mese
      │
      ├─► capienzaProgressiva(lordoMese, settings, progressivo)
      │      reddito presunto dell'anno (programma paghe) e capienza del TI
      │
      └─► nettoDelMese(lordoMese, settings, giorniMese, quota13/14, capienza)
             la busta paga del mese (calcNetMonthly sul reddito presunto)
```

`calcolaMese()` in `index.js` fa questa catena per un mese, dagli ingressi divisi per natura.

`App.jsx` calcola `payByShift` **una volta sola** con `useMemo` e la passa a valle. Ricalcolarla dentro ogni chiamata di `calcTotalPay` renderebbe l'app sempre più lenta al crescere dei turni: è `O(N)` sull'intera storia, non sul mese.

Perché il contesto è sempre `allShifts` e mai i soli turni del mese: supplementare e straordinario si contano a settimana, e una settimana a cavallo di capodanno o di fine mese va contata per intero. Passare solo il mese farebbe risultare cifre diverse fra Calendario e Statistiche per lo stesso periodo.

---

4. Fase 1 — Durata di un turno

```
calcShiftMinutes(shift)  // pay.js
```

1. Se `durationMinutes != null` → quello (assenze).
2. Altrimenti `minutesDiff(startTime, endTime) − breakMinutes`, mai negativo.

`minutesDiff()` (date.js) somma 24 h se il risultato è negativo: **è così che un turno 22:00–06:00 vale 8 ore e non −16.**

`parseTime()` ritorna `null` — non `NaN` — su un orario vuoto o malformato, e il `null` si propaga come 0. Un import da foto andato storto produce zero ore, non un totale corrotto.

---

5. Fase 2 — Raggruppamento: dove si cumulano le ore

`computePayByShift()` conta le ore in due finestre diverse, e la distinzione è il cuore del contratto mensilizzato:

CONTRATTO	SUPPLEMENTARE E STRAORDINARIO	ORE ORDINARIE PAGATE
`settings.onCall`	oltre `dailyOvertimeThreshold` nel **giorno**	le ore lavorate
CCNL **mensilizzato**	a **settimana** (lun–dom): oltre le ore del contratto, e oltre le 40	il **monte ore** del mese (24 × 4,3 = 103,20)
altrimenti	a settimana, oltre le ore del contratto	le ore lavorate

5.1 Il supplementare è della settimana, le ordinarie sono del mese

Deciso dalla busta di settembre 2026, coi turni veri, al centesimo (`check-supplementare-settimanale.mjs`). Le ore oltre le 24 del contratto in ogni settimana lun–dom sono supplementari nel giorno in cui si superano; una settimana a cavallo di due mesi resta una, e ogni ora va nel mese del suo giorno. Le ore ordinarie pagate sono invece il monte ore del mese, comunque distribuite: luglio ne aveva 109,5 lavorate, settembre 100,5 con le ferie, e la busta ne paga 103,20 tutte e due le volte. Per un mese ancora a metà la parte ordinaria sale fino al monte ore man mano che il mese si riempie (`quotaOrdinaria` in `pay.js`), senza salti.

Per un po' si era creduta mensile anche la soglia del supplementare: giugno e luglio, letti sulle ore della BUSTA, davano «ore − 103,20 = supplementari», che tornava per costruzione. Coi turni, settembre dava 11,55 h contro 14,25.

La finestra del mese è il calendario, dal 1 all'ultimo giorno: lo ha deciso la busta di agosto 2026 (`check-busta-agosto-2026.mjs`), con quindici giorni di ferie iniziati il 31 agosto e pagati per una sola giornata nel mese. `payrollMonthKey` resta in `date.js` per chi sceglie il mese di paga a settimane intere (`check-mese-paga-2026.mjs`).

5.2 Le assenze riempiono la settimana

Ferie, permessi e malattia contano per primi nella settimana: riempiono le 24 ore del contratto e non diventano mai supplementari. Quando cadono nella settimana non cambia la paga (`check-assenze.mjs`). La **festività non lavorata** invece non conta per le 24 ore: è già dentro il monte

ore del mese (contarla dava ad agosto 22,5 h contro le 17,5 stampate).

---

6. Fase 3 — Supplementare e straordinario

```
0 ─── ore del contratto (24) ─── tempo pieno (40) ──► ∞   nella settimana
   ordinarie        supplementari            straordinari
              overtimeSurchargePct    straordinarioSurchargePct
```

- `straordinarioSurchargePct === '' | null` → **eredita** `overtimeSurchargePct`.
- Lo straordinario si conta a settimana oltre le 40 ore (D.Lgs. 66/2003), anche sul mensilizzato (`check-straordinario-settimanale.mjs`).
- `onCall` **non ha** la soglia del tempo pieno: resta una sola soglia giornaliera.
- Un turno a cavallo di una soglia viene diviso con `minutesInBand` senza contarlo due volte.

6.1 Le eccezioni che governano il conteggio

(a) Le ore festive lavorate stanno FUORI dal conteggio. In busta un festivo lavorato ha una riga propria («Lavoro festivo ordinario» al 100% più «Magg. festivo») e quelle ore non compaiono fra le supplementari, né riempiono le 24 ore della settimana (`check-festivo-supplementare.mjs`). Il test usa `isHoliday(s.date, settings)`, **non** `parts.holiday`: con la maggiorazione festiva a zero quest'ultimo sarebbe zero, e le ore rientrerebbero di nascosto.

(b) Le assenze riempiono la settimana ma non possono essere eccedenti. In un giorno di ferie non si lavora, e pagarlo in più sarebbe un guadagno per essere stati assenti.

(c) Nessuna maggiorazione di giorno su un'assenza. Una domenica di ferie non prende il domenicale.

(d) Le maggiorazioni non si cumulano. Sulle ore di domenica che sono già supplementari o straordinarie resta solo l'eventuale differenza: la maggiore assorbe la minore. La domenica chiude la settimana, quindi è spesso proprio lei a passare le 24: settembre 2026 ne pagava 5,75 su 15,5. Lo **straordinario notturno** del contratto (Turismo: 60% fra le 24 e le 6, `straordinarioNotturno` in `contratti/`) sostituisce il 30% di giorno e assorbe il notturno ordinario (`check-straordinario-notturno.mjs`, dal CCNL e non da una busta).

---

7. Fase 4 — Le maggiorazioni

Sono di **due nature diverse**, e la distinzione spiega l'architettura.

7.1 Maggiorazioni di GIORNO — valgono sul turno intero

`getShiftSurchargeParts(shift, settings)` → `{ sunday, holiday, manual }`, tenute **distinte** perché il riepilogo del mese le mostra una per una: un totale unico non direbbe quale voce si scosta da quella stampata in busta.

Quando un giorno è **sia domenica sia festivo**, `holidaySundayMode` decide:

MODO	EFFETTO
`'max'` (default)	vince la maggiore, l'altra va a zero; **a parità vince il festivo** (è la ragione più specifica)
`'sum'`	si sommano
`'holiday'`	vince il festivo, il domenicale sparisce

La combinazione decide anche **a quale voce attribuire la quota**, così il riepilogo può dire quanto della paga viene da domeniche e quanto da festivi.

`isHoliday(dateStr, settings)` (holidays.js) copre: 10 festività nazionali a data fissa, **Pasqua** calcolata con il computus di Meeus/Gauss, **Pasquetta** (Pasqua + 1, con rollover di mese corretto) e il **santo patrono** opzionale in formato `'MM-DD'`. La domenica **non** è una festività: è un'altra cosa.

7.2 Maggiorazione NOTTURNA — vale sui soli minuti in fascia

Non sta in `getShiftSurchargeParts` di proposito: quella funzione restituisce percentuali sul turno intero, mentre i CCNL pagano «le ore prestate dalle 22 alle 6». Un turno 20:00–02:00 ha quattro ore notturne e due diurne.

```
minutiInFasciaNotturna(start, end, settings)  // notturno.js
```

Lavora in **minuti assoluti dall'inizio del turno**, non in ore di orologio, e confronta il turno con la finestra notturna di **tre giorni** (`-24h`, `0`, `+24h`): serve sia al turno che valica la mezzanotte sia a quello che comincia prima delle 06:00.

La fascia si risolve in quest'ordine (`fasciaNotturnaRisolta`): `settings.nightStart/nightEnd` → `ccnl.json → fasciaNotturna` → legge (22:00–06:00, D.Lgs. 66/2003). *La busta batte il contratto, il contratto batte la regola generale.* Il Turismo parte dalle 23:00, non dalle 22: un default sbagliato conta come notturne ore che in busta non lo sono e gonfia la stima verso l'alto — la direzione che fa male.

Il tetto sui minuti pagati (`minutiNotturniPagati`) esiste perché la pausa si sottrae al totale ma non si sa dove cada: senza, un 22:00–06:00 con mezz'ora di pausa darebbe 8 ore notturne su 7,5 pagate. Il tetto sta in un solo posto — `notturno.js` — perché motore e interfaccia usino la stessa regola.

Il cumulo (`pctNotturnoAggiuntiva`): la funzione restituisce il **solo supplemento** rispetto a ciò che il turno prende già. In modalità `'max'` (default, ed è ciò che scrivono Commercio e Vigilanza: «la maggiore assorbe la minore») vale `max(0, notturno − pctGiorno)`; in `'somma'` vale l'intera percentuale notturna. Con `nightSurchargePct = 0` l'intero blocco vale zero e il motore si comporta come prima che il notturno esistesse.

7.3 Attenzione: come la busta SCRIVE le percentuali

`utils/maggiorazioni.js` esiste per un problema reale, verificato su 17 cedolini: la busta usa due convenzioni e non lo dichiara.

```
«Magg. dom. 10%»          base × 0,10   →  10 è la SOLA maggiorazione
«Magg. nott. 25% P.E.»    base × 0,25   →  25 è la SOLA maggiorazione
«Supplementare 30% P.E.»  base × 1,30   →  30 è la maggiorazione (ora pagata al 130%)
«Magg. fest. 120%»        base × 1,20   →  120 è il TOTALE, cioè +20%
```

L'app chiede **sempre «quanto in più»**. `normalizzaMaggiorazione(grezzo)` sottrae 100 sopra il 100 (nessun CCNL italiano supera il 75%, quindi sopra il 100 non c'è ambiguità: è un totale), **segnala senza toccare** fra 76 e 100. Chi scrive 120 dove andava 20 si ritrova un giorno festivo gonfiato dell'83%.

---

8. Fase 5 — Assenze

`motore/assenze.js`. Quattro tipi oltre a `lavoro`: `ferie`, `permesso`, `malattia`, `festivita` (festività **non** lavorata, giustificativo a sé in busta — da non confondere con il festivo lavorato, che resta di tipo `lavoro`).

Quanto vale una giornata (`minutiGiornoAssenza`): `absenceDailyHours` se impostato, altrimenti `expectedWeeklyHours / workingDaysPerWeek` (default 6 giorni: è il caso tipico del lavoro a turni, e la busta di riferimento espone 26 giorni INPS al mese). Per un part-time 60% Turismo — 24 ore su sei giorni — fa esattamente 4 h/giorno.

Percentuale pagata (`percentualeAssenza`): - ferie e permesso → **100%** (in busta stanno *dentro* la retribuzione ordinaria, non come voce a parte); - malattia → dipende dalla **carenza**, che si conta **per evento**, non per anno. `giorniEventoMalattia(allShifts)` numera ogni giorno di malattia dentro la propria sequenza di giorni consecutivi; i primi `malattiaCarenzaGiorni` prendono `malattiaCarenzaPct`, gli altri `malattiaPct`.

La consecutività si misura con `dayNumber(iso)`, che passa da `Date.UTC`: con l'ora legale due giorni consecutivi distano 23 o 25 ore, e una differenza in millisecondi spezzerebbe la sequenza proprio lì — in silenzio, e solo sui dispositivi con fuso europeo.

> **I parametri della malattia NON sono verificati su alcun cedolino**: nessuna delle buste disponibili contiene malattia. La struttura è quella dello schema INPS, ma quanto paga davvero dipende dal CCNL. La UI deve dirlo.

---

9. Fase 6 — L'output di `computePayByShift`

Per ogni turno, una riga con 20 campi. Le voci restano **separate** perché il riepilogo del mese le confronta una a una con quelle stampate in busta.

```
{
  base,                    // minuti × rate/60 × (pctAssenza/100)
  surcharge,               // somma delle sei voci sotto
  surchargeSunday, surchargeHoliday, surchargeManual,
  surchargeOvertime, surchargeStraordinario, surchargeNight,
  overtimeBase, straordinarioBase, nightBase,   // quota di `base` per fascia
  overtimeMinutes, straordinarioMinutes, nightMinutes,
  tipo,                                          // TIPO.*
  ferieMinutes, permessoMinutes, malattiaMinutes, malattiaBase,
  ordinariPagatiMinutes,   // minuti ordinari pagati (col monte ore del mensilizzato)
  sundayMinutes,           // minuti che prendono il domenicale
  missingRate,             // nessuna paga applicabile a questa data
}
```

`overtimeBase` e `straordinarioBase` servono a **ricomporre le voci come le stampa la busta**: il cedolino non scrive il solo +30%, scrive le ore supplementari intere al 130% (`overtimeBase + surchargeOvertime`) e la retribuzione ordinaria al netto di quelle (`base − overtimeBase − straordinarioBase`). Senza questi campi le due colonne non sarebbero confrontabili.

`missingRate` non è cosmetico: senza, un turno in una data priva di paga applicabile vale 0 € e il totale è **silenziosamente** sottostimato.

9.1 La paga oraria storica

`getRateForDate(dateStr, settings)`: `settings.hourlyRate` è la paga **attuale**; `previousRates: [{until, rate}]` elenca quelle precedenti a un aumento. I turni fino a `until` **inclusa** usano quella paga. Le voci si ordinano per `until` e si prende la prima che copre la data.

`calcTotalPay` ritorna `null` se **nessuna** paga è configurata (`hasAnyRate`) — non zero: sono due cose diverse, e la UI le mostra diversamente.

> Il percorso è uno solo: `computePayByShift` → `calcTotalPay`. Una seconda strada per calcolare la paga, senza soglie né contesto, diventerebbe prima o poi un numero diverso da quello riscontrato.

---

10. Fase 7 — Dal lordo del mese al reddito annuo di riferimento

10.1 `computeAnnualGrossFromShifts(year, allShifts, settings, payByShift, oggi)`

Somma: turni dell'anno + `priorTaxableIncome` (il «montante» già maturato prima di usare l'app) + 13ª/14ª già incassate.

Il **doppio conteggio** è evitato a granularità mese: `priorIncomeDate` marca il confine, e si contano solo i turni dei mesi **successivi** a `YYYY-MM` del cutoff. Il montante si applica solo se la sua data è dello stesso anno.

Le mensilità aggiuntive si contano in base alla **data di oggi** — che passa chi chiama, come in tutto il motore — non al mese che si sta sfogliando: altrimenti aprire dicembre farebbe risultare la tredicesima già presa, cambiando reddito annuo, aliquota e soglie del bonus.

10.2 Ratei di 13ª e 14ª

`extraMonthAccrual(kind, year, settings)` → frazione 0..1. Il periodo di competenza è **luglio→giugno per la 14ª**, anno solare per la 13ª. Un mese matura il rateo solo se lavorato **almeno 15 giorni**. Senza `hireDate` si assume la mensilità piena. Con assunzione 29/12/2025, la 14ª di giugno 2026 vale 6/12 — come stampato in busta.

`EXTRA_MONTHS = { quattordicesima: 5 /*giugno*/, tredicesima: 11 /*dicembre*/ }`.

10.3 `projectAnnualIncome(...)` — quattro strade

Restituisce `{ value, source, voci[], mesiTrascorsi }`. Le `voci` sono la scomposizione mostrata nel pannello di spiegazione: **stanno qui e non nella UI** di proposito — una spiegazione che ricalcola i numeri per conto proprio prima o poi smette di combaciare con la cifra che pretende di spiegare.

PRIORITÀ	CONDIZIONE	FONTE
1	`tiProjectionMode === 'ytd'`	maturato annualizzato × 12/mesi + 13ª/14ª dell'anno
2	`annualGrossManual > 0`	scritto a mano
3	`onCall`	maturato annualizzato (non c'è contratto da cui proiettare)
4	default	**previsione in avanti**

La previsione in avanti è `annualGross + mesiRestanti × mensileDaContratto + 13ª/14ª non ancora arrivate`. Non annualizza il passato: **somma il futuro**.

La differenza non è estetica. La domanda a cui il numero deve rispondere è «accetto questo straordinario, o mi fa superare la soglia?». Con l'annualizzazione (maturato × 12/mesi-trascorsi) un euro guadagnato ad agosto ne spostava uno e mezzo, a gennaio dodici; e il vecchio `Math.max` con la proiezione da contratto faceva da **pavimento**: misurato il 21 agosto, i primi 200 € di straordinari lasciavano il margine fermo a 4.238 €, i successivi lo abbassavano di 300 € ogni 200. Un numero che non si muove e poi si muove troppo non è una risposta. Così invece **ogni euro in più sposta la previsione di esattamente un euro**.

Le 13ª/14ª **già incassate** stanno dentro `annualGross`: qui si aggiungono solo quelle che devono ancora arrivare, altrimenti la 14ª di giugno verrebbe contata due volte. Sono anche una tantum, quindi non si annualizzano mai: si annualizza la sola parte ricorrente (`recurring = annualGross − annualExtras`).

---

11. Fase 8 — `calcNetAnnual(grossAnnual, settings)`

È la **scala annuale** su cui si misura poi il mese. Ordine delle operazioni:

```
1. contributi   = calcContributi(gross, settings, mensileDaContratto × 12)
2. imponibile   = gross − contributi.deducibili + contributi.fringeImponibile
3. IRPEF lorda  = scaglioni su imponibile
4. detrazioni   = detrazioneLavoro(imponibile) + detrazioneCuneo(imponibile)
5. IRPEF netta  = max(0, lorda − detrazioni)
6. addizionali  = imponibile × aliquote, SOLO se irpefNetta > 0
7. TI           = trattamentoIntegrativo(imponibile, lorda, detLav, detrTot)
8. cuneo        = imponibile × cuneoPercent(imponibile)
9. net          = gross − contributi − irpefNetta − addizionali + TI + cuneo
```

11.1 Contributi (`calcContributi`)

Tre cose che non si indovinano e sono state lette dalle buste:

1. **L'imponibile previdenziale è arrotondato all'EURO**, e le aliquote si applicano a quello. Turismo luglio 2026: lordo 1.173,48 → IVS su 1.173,00 = 107,80 (sul lordo pieno uscirebbe 107,84). Fiduciari giugno: 756,77 → IVS su 757,00 = 69,57 (contro 69,55).
2. **L'Ente Bilaterale non si calcola sul lordo** ma sulla retribuzione *contrattuale* (minimo tabellare + contingenza riproporzionati al part-time) — `settings.ebtBase`, leggibile dalla busta — e **non è deducibile**.
3. **La quota Ente Bilaterale a carico DITTA** non viene trattenuta ma è un **fringe benefit** che si somma all'imponibile fiscale, ed è **troncata** a due decimali: 948,05 × 0,20% = 1,8961 → 1,89.

Il terzo elemento, e perché la base dell'Ente Bilaterale non è la mensilità. La paga oraria della busta di riferimento si compone così:

```
minimo tabellare   1.057,72
contingenza          522,37
terzo elemento         5,41   ← voce a sé, stampata in busta
                  ──────────
mensile full-time  1.585,50   ÷ 172 = 9,21802 €/h   × 60% = 951,30 €
```

Il **terzo elemento** è un importo fisso della contrattazione territoriale: entra nella retribuzione — quindi nella paga oraria, quindi nei 951,30 — ma **non** nella base dell'Ente Bilaterale, che si ferma alle prime due voci (948,05 = (1.057,72 + 522,37) × 60%). Da lì i **3,25 €** di scarto fra le due, rimasti a lungo annotati come inspiegati. Riscontro: `scripts/check-tabellare-turismo.mjs`.

Senza `ebtBase`, il motore ripiega sulla mensilità da contratto e quindi **sovrastima la base di quei 3,25** — scelta consapevole: valgono lo 0,2% di 3,25, cioè meno di un centesimo di trattenuta e un centesimo sull'imponibile attraverso la quota ditta. Ricostruirla esatta vorrebbe dire chiedere il terzo elemento in Impostazioni: un campo in più per meno di un centesimo, non si fa. `ebtBase` resta l'appiglio per i riscontri, che il numero preciso lo leggono dal cedolino; in Impostazioni non c'è, di proposito.

FIS e CIGS non sono parametri del CCNL (`motore/contributi-legge.js`). L'aliquota dipende da **quanti dipendenti ha l'azienda** — un bar con quattro persone e una catena con duecento applicano lo stesso CCNL Turismo e pagano contributi diversi. Del contratto dipende solo *a quale fondo* si è iscritti (`ammortizzatori: 'fis'`).

	FINO A 5	6–15	OLTRE 15
FIS totale	0,50%	0,80%	0,80%
FIS a carico lavoratore	⅓ del totale	⅓	⅓ (= 0,2667%)
CIGS a carico lavoratore	—	—	0,30%

Default `'oltre15'`: è la fascia delle due buste verificate ed è anche il valore che riproduce il comportamento precedente dell'app.

11.2 Parametri fiscali per anno (`fisco/`)

Un file per anno (`fisco/2026.js`), scelto da `fiscoDellAnno(anno)`: l'anno noto più recente fino a quello chiesto. Chi conosce l'anno — netto del mese, conguaglio, rischio di restituzione, `calcolaMese` — mette la tabella nelle impostazioni con `conFisco(impostazioni, anno)`; le funzioni la leggono con `fiscoDi(impostazioni)`, e chi non dice l'anno ha l'ultima nota (`check-fisco-anno.mjs`). Il 2026:

```
IVS dipendente               9,19%
IRPEF                        23% ≤28.000 · 33% ≤50.000 · 43% oltre   (L. 199/2025)
Detrazione lav. dip.         1.955 fino a 15.000
                             1.910 + 1.190×(28.000−R)/13.000   fra 15.000 e 28.000
                             1.910×(50.000−R)/22.000           fra 28.000 e 50.000
                             +65 € fra 25.000 e 35.000
Trattamento integrativo      1.200 fino a 15.000 (se capiente) · a scalare fino a 28.000
   capienza                  irpefLorda > detrazioneLavoro − 75
Cuneo L. 207/2024 (≤20.000)  7,1% ≤8.500 · 5,3% ≤15.000 · 4,8% ≤20.000
Detrazione cuneo             1.000 € fra 20.000 e 32.000, a scalare fino a 40.000
Addizionali default          regionale 1,23% · comunale 0%
```

Le soglie sono definite sul **reddito complessivo**, non sul lordo: per un dipendente è il lordo al netto dei contributi (art. 51 TUIR; riscontro su una CU in `check-cu-2025.mjs`). Il lordo che corrisponde a una soglia lo dà `lordoPerReddito()` (`motore/soglia-lorda.js`): l'ultimo euro lordo che resta entro il reddito, cercato col reddito vero (`redditoComplessivo()`). Dipende dal contratto: senza CCNL ~16.518 € e ~30.833 €, nel Turismo ~16.621 € e ~31.027 €. A schermo si dice prima il lordo, da confrontare con la busta, e i 15.000 tra parentesi: «La tua soglia lorda è ~16.621 € l'anno (15.000 € di reddito)». Fondo pensione e cassa sanitaria trattenuti in busta (`motore/previdenza.js`) entrano in `calcContributi` come contributi deducibili: abbassano il reddito e alzano la soglia lorda (1,5% al fondo nel Turismo: da ~16.596 a ~16.876 €).

Lo sconto di 75 € nella capienza del TI **non è riscontrato su busta**: viene dalla norma. Nelle buste disponibili la capienza c'è comunque, con o senza. Se un cedolino dicesse il contrario, **vince il cedolino**.

---

12. Fase 9 — `calcNetMonthly(...)` — la busta del mese

```
nettoDelMese(lordoMese, settings, giorniMese, extraMese, capienza)
  → calcNetMonthly(lordoMese, riferimentoAnnuoDelMese(...), settings, giorniMese, extraMese,
capienza)
```

Il riferimento annuo è il REDDITO PRESUNTO dell'anno, non il lordo del mese × 12: quanto pagato finora più questo mese per i mesi che restano e per la 13ª (`lordoPresuntoAnno` del programma paghe, in `programmi-paghe/`). Da lì il datore decide in blocco detrazione, indennità e trattamento integrativo, e le detrazioni di agosto e settembre 2026 tornano al centesimo (`check-reddito-presunto.mjs`). Senza progressivo decide la regola del programma paghe per quel caso (`annoSenzaProgressivo`: il lordo × 12).

Trattenute e bonus sono voci separate: il bonus non riduce le trattenute.

```
contributi   = calcContributi(monthGross, settings, mensileDaContratto)
imponibile   = round2(gross − deducibili + fringe)

── binario ordinario ──                    ── binario 13ª/14ª ──
imponibileOrdinario                        imponibileExtra = extra × (1 − aliqDeducibile)
irpefLordaOrd = ann.irpefLorda × ratio     irpefExtra = imponibileExtra × aliquotaMarginale
  ratio = imponibileOrd / ann.imponibile     (nessuna detrazione: va a conguaglio)

detrazioni   = (detrLav + detrCuneo) × monthDays/365
irpefNetta   = max(0, irpefLordaOrd − detrazioni) + irpefExtra
addizionali  = imponibile × aliquote, se ann.irpefNetta > 0
trattenute   = round2(contributi + irpefNetta + addReg + addCom + trattenuteFisse)

TI           = quota(ann.TI × monthDays/365)       quota: del programma paghe
cuneo        = quota(imponibile × cuneoPercent(ann.imponibile))
tfr          = tfrLordo − tfrImposta          (opzionale, tassazione separata)

net = round2(gross − trattenute + TI + cuneo + tfr)
```

Quattro dettagli che vengono dalle buste e non dalla teoria:

1. **Le detrazioni si rapportano ai GIORNI del mese** (`monthDays/365`), non alla quota di imponibile: è così che le calcola il sostituto d'imposta. Verificato: 1.955 × 31/365 = **166,04**.
2. **La mensilità aggiuntiva viaggia su un binario fiscale separato** («tassazione autonoma»): aliquota **marginale**, e **non assorbe detrazioni**.
3. **Il TI è una quota annua spalmata sui giorni; il cuneo NO.** L'indennità L. 207/2024 in busta è la percentuale di fascia applicata all'imponibile **del mese**, quindi segue le ore effettivamente lavorate. Verificato: 4,8% × 1.849,65 = 88,78.
4. **`detrazioniApplicate`** esiste solo per la UI: quando la detrazione supera l'IRPEF lorda il netto è già giusto, ma scrivere «lorda 157 − detrazioni 161 = netta 0» sembra un errore di conto. In busta compare la parte capiente.

12.1 Arrotondamenti

```
round2 = (x) => Math.round(x * 100) / 100;      // net.js
quota  = (x) => Math.floor(x * 100 + 1e-9) / 100; // programmi-paghe/zucchetti.js
```

Non è un dettaglio estetico: **il cedolino chiude ogni voce a due decimali e le somme partono da quelle**. Alcune voci il programma paghe le **tronca** invece di arrotondarle — trattamento integrativo, indennità e quota Ente Bilaterale ditta: 1.200 × 31/365 = 101,9178 in busta è **101,91**, non 101,92. Come si chiude al centesimo è una regola del programma paghe, e sta nel suo file (`check-programma-paghe.mjs`).

12.2 Il TFR in busta

Opzionale (`tfrInBusta`). Quota che matura sul lordo: `1/13,5 − 0,50%` ≈ 6,91%, soggetta a **tassazione separata** (default 23%, `tfrTaxRate` per cambiarla) — a differenza di TI e cuneo, che sono esenti, qui l'imposta va sottratta.

> **Nota aperta sulla BASE del TFR.** Le due buste disponibili la calcolano in due modi diversi e nessuno dei due è il lordo pieno che il motore usa (fiduciari: sola retribuzione ordinaria; Turismo: retribuzione + indennità «utili al TFR» + maggiorazione domenicale, ma senza supplementare — e nemmeno così torna, 67,39 contro i 66,39 stampati). Due

> buste che si contraddicono e una che non quadra non bastano a stabilire una regola: la base resta il lordo finché non arriva un terzo cedolino. Non tocca il netto in nessuno dei casi.

---

13. Cosa è verificato e cosa no

Distinzione essenziale: chi modifica il motore deve sapere quali numeri sono **fatti letti su un cedolino** e quali sono **inferenze da norma**.

Riscontrato al centesimo su buste reali - arrotondamento all'euro dell'imponibile INPS (due buste, due CCNL diversi); - aliquote IVS/FIS/CIGS/Ente Bilaterale, base e troncamento della quota ditta; - detrazioni rapportate ai giorni, IRPEF lorda proporzionale, ritenute; - TI troncato sui giorni, cuneo sull'imponibile del mese; - supplementare a settimana e ore ordinarie pari al monte ore (settembre 2026, coi turni veri); domenicale che non si somma al supplementare; - reddito presunto dell'anno con la 13ª: detrazioni di agosto e settembre 2026; - mese di calendario come finestra del mese (agosto 2026); - esclusione delle ore festive dal conteggio delle eccedenze; - rateo 6/12 della 14ª con assunzione 29/12/2025; - maggiorazioni Turismo su 17 cedolini: notturno 25%, domenicale 10%, supplementare 30%, festivo 20% (scritto «120%» in busta); - **composizione della paga oraria**: tabellare + contingenza + terzo elemento ÷ 172, e il terzo elemento fuori dalla base dell'Ente Bilaterale (§11.1).

Non riscontrato — struttura da norma, valori da confermare - **malattia**: carenza e percentuali. Tre cedolini la contengono, ma nessuno fa testo: i due del 2024 sono di un datore che **integrava** la malattia per scelta aziendale, non per obbligo di CCNL, e quello di marzo 2026 la riporta solo come storno delle ore. Una busta che documenta la generosità di un'azienda non è una regola da mettere in un motore (vedi `CLAUDE.md`); - sconto di 75 € nella capienza del TI, e il TI concesso il primo mese; - **straordinario** oltre le 40 ore e **straordinario notturno** al 60% (norma e CCNL: nessuna busta ha una settimana oltre le 40); - **fascia notturna del Turismo**: le buste non riportano le timbrature, e l'art. 13 prevede orari diversi per settore (24:00–06:00, 23:00–06:00, 23:30–06:30); - **base del TFR** (vedi §12.2).

In `contratti/`, solo i file con `verificato: true` sono stati riscontrati voce per voce. Un contratto che non ha un file vale `LEGGE`; i nomi degli altri (1.163 nell'archivio CNEL) stanno all'app, non al motore.

---

14. Come si verifica una modifica

I riscontri girano su **Node puro** e importano gli stessi moduli dell'app. Non serve build.

```
node src/motore/riscontri/check-numeri-fermi.mjs     # l'uscita non si è mossa di un centesimo
node src/motore/riscontri/check-confine-motore.mjs   # niente import da fuori, niente orologio
npm run riscontri                                    # tutti, anche con l'orologio spostato
```

Quelli delle **regole** stanno in `src/motore/riscontri/` e viaggiano col motore; quelli sulle **buste** di una persona stanno nell'app che li ha (in Turni `scripts/check-busta-*.mjs`, `check-supplementare-settimanale.mjs`, `check-reddito-presunto.mjs`…), perché contengono le sue cifre.

Una modifica al motore che fa fallire uno di questi riscontri è sbagliata finché non si dimostra il contrario con una busta alla mano. Uno spostamento di codice non deve muovere i numeri fermi; un cambio di regola li muove, e allora si riscrivono con `--aggiorna` nello stesso commit che lo dichiara.

Output atteso di `check-busta-luglio-2026.mjs` (CCNL Turismo, part-time 60%, paga 9,21802 €/h, 31 giorni):

```
Imponibile INPS (arrotondato all'euro)  1173.00
IVS 9,19%                                107.80
Imponibile fiscale                      1060.92
IRPEF lorda (23%)                        244.01
Detrazioni lav.dip. (1.955 × 31/365)     166.04
Ritenute IRPEF                            77.97
Trattamento integrativo                  101.91
Indennità L.207/2024 (5,3%)               56.22
NETTO DEL MESE                          1137.29
```

---

15. Trappole note

Le cose che si rompono per prime, con il sintomo.

MODIFICA APPARENTEMENTE INNOCUA	COSA SI ROMPE
togliere l'estensione `.js` da un import interno	i riscontri smettono di partire (`ERR_MODULE_NOT_FOUND`)
contare il supplementare sul mese invece che sulla settimana	settembre 2026: 11,55 h invece di 14,25
contare le ore festive o la festività non lavorata nelle 24 della settimana	supplementari in più di quante la busta ne paghi
sommare il domenicale al supplementare sulle stesse ore	settembre 2026: 15,5 h di domenicale invece di 5,75
leggere `new Date()` dentro il motore	un riscontro diventa rosso il primo del mese; il motore non si porta via
importare da fuori `src/motore`	il motore non si copia più da solo (`check-confine-motore`)
usare `parts.holiday` invece di `isHoliday` per il test festivo	con maggiorazione festiva a 0 le ore rientrano fra le supplementari
calcolare i contributi sul lordo esatto	4 centesimi di scarto per voce, su ogni busta
arrotondare invece di troncare TI/cuneo/quota ditta (programma paghe)	1 centesimo di scarto, ripetuto
applicare le detrazioni in proporzione all'imponibile	detrazione sbagliata in ogni mese ≠ 30,4 giorni
passare al motore i soli turni del mese	settimane a cavallo spezzate: Calendario ≠ Statistiche
ricalcolare `computePayByShift` a ogni chiamata	l'app rallenta linearmente con la storia dei turni

MODIFICA APPARENTEMENTE INNOCUA	COSA SI ROMPE
trattare `''` come `0` in un campo «vuoto = calcolato»	comportamento cambiato senza che l'utente tocchi nulla
sottrarre la pausa ai minuti notturni	doppia sottrazione: motore e UI divergono
confrontare date con differenze in millisecondi	ora legale: 29 marzo e 25 ottobre 2026 spezzano le sequenze
dare al Turismo la fascia notturna 22:00–06:00	ore contate come notturne che in busta non lo sono
impostare 120 come maggiorazione festiva	giorno festivo gonfiato dell'83%

---

16. Due grandezze che possono legittimamente non coincidere

Non sono bug, e vanno capite prima di «correggerle»:

- **Ore dell'app vs ore in busta.** La busta paga il monte ore del mese (103,20 h) anche quando le ore ordinarie lavorate sono di più o di meno, e include ferie, permessi e festività non lavorate. Se non sono state segnate nell'app, il totale dell'app resta sotto. È lo scarto per cui esistono `motore/assenze.js`, `motore/periodo-assenza.js` e `utils/festivita-non-lavorate.js`.
- **Mese di calendario vs mese di paga.** La busta usa il calendario (agosto 2026); il mese di paga a settimane intere resta una scelta del Calendario di Turni, dichiarata sopra i numeri quando non coincide col mese visualizzato.
- **Un mese di quattro settimane esatte** arriva al massimo a 96 h ordinarie: se le supplementari non bastano a coprire il resto, l'app mostra meno delle 103,20 che la busta paga comunque (`COSE-NUOVE.md`, J3).

---

17. Il gate beta

`src/config/features.js` → `ENABLE_NET_CALC` (default ON, spegnibile con `VITE_BETA_NET=false`) governa **tutto il motore fiscale**. Con il flag OFF l'app calcola ore e lordo ma non mostra nulla del netto. `useMonthlyNet` legge la costante direttamente; `stats.monthlyBreakdown` e `projectAnnualIncome` la ricevono come parametro (`enableNetCalc`) e azzerano i termini che dipendono dal contratto. Chiamare il motore con `enableNetCalc: false` deve dare gli stessi numeri di un'app senza motore fiscale.

---

18. Se devi estendere il motore

1. **La busta batte la norma, la norma batte l'intuizione.** Ogni costante nuova va accompagnata dal cedolino su cui è stata letta, o da una nota esplicita che dice che non è verificata.
2. **Un dato nuovo va nel posto giusto:** dipende dal contratto → il suo file in `contratti/`; dipende dall'anno → `fisco/`; è una scelta del programma paghe → `programmi-paghe/`; dipende

dalla legge e dall'azienda → `contributi-legge.js`; dipende dal lavoratore (livello, classificazione) → è un'impostazione, e va scritta in `impostazioni.js`.

3. **Percentuali sul giorno** → `getShiftSurchargeParts`. **Percentuali su una fascia oraria** → un modulo proprio, come `notturno.js`, che restituisce il solo supplemento.
4. **Voci separate, sempre.** Un totale unico non permette di confrontare riga per riga con la busta, ed è l'unico modo di scoprire dove il motore sbaglia.
5. **Scrivi lo script di riscontro prima di cambiare la formula**, e verifica che fallisca per la ragione giusta.
6. **Il default deve riprodurre il comportamento precedente.** Ogni funzione aggiunta al motore (notturno, straordinari, contributi minori) ha un valore di partenza che la spegne: chi non tocca il campo nuovo vede gli stessi numeri di prima.
7. **Il motore non esce dalla sua cartella e non legge l'orologio.** Una data di oggi si riceve (`richiediOggi`), un dato del browser non si legge mai.

PARTE 6

I riscontri

Cosa si sa e come lo si sa

Del motore — src/motore/riscontri

Regole verificate su dati sintetici o sulla norma; viaggiano col motore e girano con `npm test` nella sua cartella.

`check-addizionali.mjs`	Quando scattano le addizionali regionale e comunale — e perché sono un
`check-assenze.mjs`	Riscontro di ferie, permessi e malattia.
`check-bonus.mjs`	Riscontro delle soglie del trattamento integrativo (ex bonus Renzi):
`check-confine-motore.mjs`	Il CONFINE del motore: `src/motore/` non importa niente da fuori.
`check-costo-soglia.mjs`	Quanto costa DAVVERO superare i 15.000 € — il numero su cui si decide se
`check-fisco-anno.mjs`	Il fisco dell'ANNO arriva davvero ai calcoli:
`check-impostazioni-motore.mjs`	Le impostazioni del motore sono TUTTE scritte in impostazioni.js:
`check-lordo-turno.mjs`	Il lordo del singolo turno, riscontrato con:
`check-notturno.mjs`	Riscontro del lavoro notturno, da eseguire con:
`check-numeri-fermi.mjs`	I NUMERI FERMI: l'uscita del motore su casi fissi, confrontata al centesimo
`check-periodo-assenza.mjs`	Riscontro delle assenze a periodo, da eseguire con:
`check-premio-risultato.mjs`	Il bonus come premio di risultato, tassato a parte:
`check-previdenza.mjs`	Fondo pensione e cassa sanitaria trattenuti in busta, riscontrati sulla norma:
`check-programma-paghe.mjs`	Il programma paghe è DAVVERO un file a sé:
`check-proiezione.mjs`	Riscontro della previsione di reddito annuo:
`check-restituzione.mjs`	Il rischio di restituzione del trattamento integrativo, riscontrato con:
`check-straordinario-notturno.mjs`	Lo straordinario notturno del CCNL Turismo:
`check-straordinario-settimanale.mjs`	Lo straordinario si conta a SETTIMANA, anche sui contratti mensilizzati:
`check-ti-capienza.mjs`	Sotto i 1.250 € al mese il trattamento integrativo NON spetta sempre:

Di Turni — scripts/

I riscontri sulle buste paga di una persona e quelli dell'interfaccia. Restano all'app perché contengono le sue cifre; `npm run riscontri` li fa girare tutti, anche con l'orologio spostato a gennaio, al 31 dicembre e all'anno dopo.

`check-aggiornamento.mjs`	Il service worker non deve ricaricare la pagina addosso a chi la sta usando:

`check-anteprima-link.mjs`	L'anteprima del link condiviso in chat, riscontrata sul PACCHETTO:
`check-avvisi.mjs`	«Un avviso alla volta», riscontrato invece che sperato:
`check-backup.mjs`	«Un ripristino non lascia i dati a metà», riscontrato invece che sperato:
`check-bozza.mjs`	«Un tocco fuori non butta via il lavoro», riscontrato invece che sperato:
`check-busta-agosto-2026.mjs`	Riscontro della busta di agosto 2026 (CCNL Turismo, livello 5, part-time 60%):
`check-busta-giugno-2026-fiduciari.mjs`	Riscontro del motore netto contro una busta paga REALE, secondo contratto.
`check-busta-giugno-2026.mjs`	Riscontro del motore netto contro una busta paga REALE.
`check-busta-luglio-2026-fiduciari.mjs`	Riscontro del motore netto contro una busta paga REALE, luglio 2026.
`check-busta-luglio-2026.mjs`	Riscontro del motore netto contro la busta paga REALE di luglio 2026.
`check-busta-maggiorazioni-reali.mjs`	Le percentuali di maggiorazione, misurate su tre anni di cedolini veri:
`check-busta-mensilita-aggiuntive.mjs`	Riscontro delle MENSILITA' AGGIUNTIVE contro tre cedolini reali:
`check-buste-2026.mjs`	Riscontro del motore del netto contro TUTTE le buste 2026 disponibili:
`check-codice-morto.mjs`	Codice che non raggiunge più nessuno:
`check-configurazione.mjs`	I file che non sono né JavaScript né CSS, e i contratti che li legano:
`check-confronto-montante.mjs`	Riscontro dell'avviso «il montante non torna coi turni»:
`check-conguaglio.mjs`	Riscontro della stima del conguaglio di fine anno:
`check-consegna-file.mjs`	Riscontro della strada scelta per consegnare un file all'utente:
`check-correzione-orari.mjs`	Riscontro della correzione degli orari a :50, da eseguire con:
`check-cosa-cambia.mjs`	Riscontro del modulo cosa-cambia.js
`check-cu-2025.mjs`	Cos'è il «reddito complessivo» dei 15.000, deciso su una Certificazione Unica:
`check-dati-in-uscita.mjs`	Che cosa lascia il dispositivo:
`check-dimensioni.mjs`	Nessun file cresce senza limite:
`check-documentazione.mjs`	La documentazione cita solo cose che esistono:
`check-errore-import.mjs`	Riscontro delle due famiglie di errore dell'import da foto:
`check-festivita.mjs`	Riscontro della festività non lavorata:
`check-festivo-supplementare.mjs`	Le ore lavorate in un giorno festivo NON sono ore supplementari:
`check-import-turni.mjs`	Cosa entra nell'archivio da un import da foto:

`check-impostazioni.mjs`	«Il pulsante Salva o salva, o dice perché», riscontrato invece che sperato:
`check-installazione.mjs`	Riscontro dell'avviso «aggiungila alla Home» su iOS:
`check-leggibilita.mjs`	«Il testo colorato si legge», riscontrato invece che sperato:
`check-magg-busta.mjs`	Riscontro delle percentuali di maggiorazione:
`check-mese-paga-2026.mjs`	Riscontro dell'attribuzione delle ore al MESE DI PAGA per i CCNL
`check-montante-mensilita.mjs`	Riscontro del montante davanti alle mensilita' aggiuntive:
`check-netto-coerente.mjs`	Il netto dello stesso mese, uguale in Calendario e in Statistiche:
`check-orari-proposti.mjs`	Gli orari che il modulo propone da solo, riscontrati con:
`check-orario-cella.mjs`	Riscontro del formato corto degli orari nella griglia del mese:
`check-primo-avvio.mjs`	Riscontro delle regole del primo avvio:
`check-proxy-difese.mjs`	Riscontro delle difese del proxy AI, da eseguire con:
`check-reddito-presunto.mjs`	Il reddito PRESUNTO dell'anno: su cosa il datore decide, ogni mese,
`check-serie-giorni.mjs`	Riscontro delle serie di giorni lavorati consecutivi.
`check-sposta-orario.mjs`	«Mezz'ora avanti, mezz'ora indietro», riscontrato:
`check-supplementare-settimanale.mjs`	Il supplementare si conta a SETTIMANA, le ore ordinarie sono il MONTE ORE:
`check-swipe.mjs`	«Scorrere di lato cambia mese, e nient'altro lo fa», riscontrato:
`check-tabellare-turismo.mjs`	La tabella retributiva del CCNL e la voce «Retribuzione» del cedolino:
`check-ti-mensile.mjs`	Il trattamento integrativo, deciso mese per mese come fa la busta:
`check-verifica-busta.mjs`	La pagina «Confronta con la busta» mantiene quello che promette: